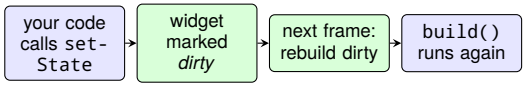
Creating interactive applications

Worked examples: the sum app and the stopwatch (`code/week05`)

Key ideas

- **Interactive app**: the user types or taps *while it runs*, and the screen answers. Built with stateful widgets.
- Snippet: type `stful` (VS Code) or `stf` (Android Studio); the editor writes both classes.
- The State class name starts with _: private to its file.

What setState really does



- **Never call** `build` **yourself**; only call `setState`.
- Keep the function short: the next frame waits for it.
- `_count++; setState(() {});` works too, but put the change *inside*: clearer.
- Change a variable *without* `setState`: the screen keeps the old value until the next rebuild.

Life cycle of a State

Method	When / use it to
`initState`	once, at the start: start a `Timer`, create a controller. No `Theme.of` here
`didChangeDependencies`	after `initState`, and when `Theme` or `MediaQuery` change
`didUpdateWidget`	parent gives new parameters: compare `oldWidget` and `widget`
`build`	required; after each of the above and each `setState`
`reassemble`	hot reload only
`deactivate`	removed from the tree
`dispose`	once, at the end: cancel timers, dispose controllers

Order: start `initState` → `didChangeDependencies` → `build`; end `deactivate` → `dispose`. Example `09_lifecycle_log` prints the order.

Forgetting `dispose` for a `Timer` or a stream: it keeps running after the widget is gone (memory leaks, crashes).

TextField

`style`	`TextStyle` of the typed text
`decoration`	`InputDecoration` (singular!)
`onChanged`	after *every* letter, with the text
`onSubmitted`	*once*: Enter, or the "done" key
`keyboardType`	`TextInputType.number`, `.emailAddress`
`controller`	read/change the text from code
`obscureText`	`true`: a password
`maxLength`	maximum characters

`InputDecoration`	Shows
`border`	`OutlineInputBorder()`: a frame
`hintText`	grey text while empty
`labelText`	label that moves up when typing
`prefixIcon`, `suffixIcon`	icon left / right
`helperText`	small help under the field
`errorText`	red message; `null` = none

A `TextField` needs a `Scaffold` (Material) above it, and a width (`SizedBox` or `Expanded`) inside a `Row`.

The text follows the keyboard

```
String _text = '';
void updateText(String text) {
  setState(() { _text = text; });
}
// in build:
Text('You typed: $_text'),
TextField(onChanged: (text) {
  updateText(text);
}),
```

letter typed → `onChanged` → `updateText` → `setState` → `build`.

Copyrighted content Dr Mohamad AOUDE

Your own field with a callback

```
class MyTextField extends StatelessWidget {
  const MyTextField({super.key,
    required this.hint,
    required this.onChanged});
  final String hint;
  final ValueChanged<String> onChanged;
  @override
  Widget build(BuildContext context) =>
    SizedBox(width: 300, height: 50,
      child: TextField(
        decoration: InputDecoration(
          border: const OutlineInputBorder(),
          hintText: hint),
        onChanged: onChanged));
}
// the page owns the data:
MyTextField(onChanged: updateText,
            hint: 'Enter some text')
```

`VoidCallback`	no parameter: `onPressed`
`ValueChanged<T>`	one value: `onChanged`

`onChanged: updateText` *gives* the function. `onChanged: updateText()` *calls* it during build: "setState() called during build".

Several files: libraries

`main.dart`	only starts the app; imports `home.dart`
`home.dart`	the page; imports `my_text_field.dart`
`my_text_field.dart`	the reusable field

- Every `.dart` file is a **library**. `import 'home.dart';` is relative to the current file; SDK and packages use `package:`.
- Each file imports what *it* uses.
- `_name` is private to its *file*, not its class.
- `show A`, `hide A`, `as a` (then `a.A`).
- File names in `snake_case`; one main class per file.

The sum app

```
double? _x, _y;   // null = empty or not a number
void updateX(String x) =>
    _x = double.tryParse(x.trim()); // no setState
void updateText() {
  setState(() {
    final x = _x, y = _y;
    _text = (x == null || y == null)
        ? 'Please fill all fields'
        : (x + y).toString();
  });
}
```

`updateX` needs no `setState`: nothing on screen changes until **SUM**. $1 + 3$ shows `4.0` (a `double`).

Read numbers safely

`double.parse('abc')`	crash: `FormatException`
`double.tryParse('abc')`	`null`: test it

```
keyboardType: TextInputType
  .numberWithOptions(decimal: true, signed: true),
String? _errorOf(String t) =>
  t.trim().isEmpty ? null
  : double.tryParse(t.trim()) == null
      ? 'Not a number' : null;
onPressed: ready ? _sum : null, // disabled
```

A numeric keyboard is a hint, not a check.

TextEditingController

```
final _controller =
    TextEditingController(text: 'Flutter');
@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
TextField(controller: _controller)
_controller.text;          // read
_controller.text = 'abc';  // change
_controller.clear();       // empty
```

A change made by code does **not** call `onChanged`: call `setState` yourself if the screen depends on it.

More buttons

`FilledButton`	filled, no shade (Material 3)
`IconButton`	tappable icon
`FloatingActionButton`	round, floats over the page
`DropdownButton`	a list of choices

`shape` in `styleFrom`:

`StadiumBorder()`	a pill
`RoundedRectangleBorder(..)`	rounded corners
`CircleBorder()`	a circle (icon + padding)

Also: `padding`, `minimumSize`.

```
style: ButtonStyle(backgroundColor:
  WidgetStateProperty.all<Color>(Colors.green)),
```

A value per state (pressed, hovered, disabled). With `.all` a disabled button stays green; `styleFrom` keeps it grey. Old name: `MaterialStateProperty`.

Stopwatch: state that changes by itself

```
Timer? _timer;          // null until Start
bool _isTicking = false;
void _startTimer() {
  _timer = Timer.periodic(
      const Duration(seconds: 1), _onTick);
  setState(() { _seconds = 0;
                _isTicking = true; });
}
void _onTick(Timer t) {
  if (mounted) setState(() => ++_seconds);
}
void _stopTimer() {
  _timer?.cancel();
  setState(() => _isTicking = false);
}
@override
```

Copyrighted content Dr Mohamad AOUDE

```
void dispose() { _timer?.cancel();
                 super.dispose(); }
```

```
ElevatedButton(onPressed:
  _isTicking ? null : _startTimer, ...)
TextButton(onPressed:
  _isTicking ? _stopTimer : null, ...)
```

- `mounted`: `false` once the widget left the tree.
- `late Timer` never assigned ⇒ `LateInitializationError` in `dispose`: use `Timer?`.
- A `Timer` drifts; for precision use Dart's `Stopwatch`.
- Big `build`? Split it: `_buildCounter()`, `_buildControls()`, ...
- Theme style: `textTheme.headlineSmall` (was `headline5`).

Scrolling

```
ListView(children: [
  for (final lap in _laps)   // collection-for
    ListTile(title: Text('$lap')),
])
```

A `Column` overflows; a `ListView` scrolls.

`physics:`	Feel
`BouncingScrollPhysics()`	bounces (iOS)
`ClampingScrollPhysics()`	stops (Android)
`NeverScrollableScrollPhysics()`	no finger scroll

Keep the platform's default unless you have a reason.

`ListView.separated`	divider between rows
`GridView`	grid of tiles
`SingleChildScrollView`	one big child: a form
`PageView`	swipe page by page
`CustomScrollView`	slivers, collapsing bar

Common errors

Screen does not change	change data in `setState`
setState() called after dispose()	`if (!mounted) return;`
No Material widget found	put the page in a `Scaffold`
InputDecorator ... unbounded width	`TextField` in a `Row`: `Expanded` or `SizedBox`
RenderFlex overflowed ... on the bottom	keyboard open: `SingleChildScrollView`
FormatException	`tryParse`, test `null`

Copyrighted content Dr Mohamad AOUDE